- **具名、匿名函数**
  - 函数分为具名函数和匿名函数，具名函数是指函数要有具体的名称，匿名函数是指没有名字的函数。
  - 具名函数

```
// 具名函数 有名字
function foo() {
    console.log('foo 函数被执行...');
}
foo(); // 调用 foo 函数
```

  - 匿名函数

```
// 匿名函数
let bar = function () {
    console.log('声明函数时没有名字...');
}
```

    - 将函数赋值给变量，即函数表达式。
    - 函数表达式和普通函数并无本质上的区别：

```
// 函数表达式与普通函数本质上是一样的
let counter = function (x, y) {
    return x + y
}
// 调用函数
let result = counter(5, 10)
console.log(result)
```

      - 普通函数的声明与调用无顺序限制，推荐做法先声明再调用
      - 函数表达式必须要先声明再调用

- **回调函数**
  - 如果将函数 A 做为参数传递给函数 B 时，我们称函数 A 为回调函数
  - 常见的使用场景：

```
function fn() {
    console.log('我是回调函数...')
}
// fn传递给了setInterval，fn就是回调函数
setInterval(fn, 1000)
```

- 定时器还有一个常见写法：

```
// 调用定时器，匿名函数做为参数
setInterval(function () {
    console.log('我是回调函数...')
}, 1000);
```

- 最简单的回调函数：

```
let fn = function () {
    console.log('我后打印');
}

function getSum(callback) {
    console.log('我先打印')
    callback()
}
getSum(fn)
```

- **环境对象**
  - 环境对象指的是函数内部特殊的变量 this ，它代表着当前函数运行时所处的环境
  - 函数的调用方式不同，this 指代的对象也不同
  - 【谁调用， this 就是谁】 是判断 this 指向的粗略规则
  - 直接调用函数，其实相当于是 window.函数，所以 this 指代 window

- **事件监听**
  - 什么是事件监听?
    - 就是让程序检测是否有事件产生，一旦有事件触发，就立即调用一个函数做出响应
  - 语法：

```
元素.addEventListener(事件类型, 事件触发时调用的回调函数)
```

- 事件监听三要素：
  - 事件源： 哪个dom元素被事件触发了，要获取dom元素
  - 事件类型： 用什么方式触发，比如鼠标单击 click、鼠标经过 mouseover 等
  - 事件调用的回调函数： 要做什么事

举例说明

```
<button button id="btn"> 唐伯虎 </button>
<script>
    // 1. 获取dom元素
    let btn = document.getElementById('btn')
    // 2. 添加事件监听
    btn.addEventListener('click', function () {
        alert('点秋香')
    })
</script>
```

注意：事件类型要加引号

- 事件中的this
  - 任何函数内都有this，事件触发时的回调函数里也不例外
  - 事件回调函数里的 this 指向的是 当前被添加事件监听的DOM元素对象
  - 简单理解【给哪个元素调用的addEventListener，回调函数里的this就是哪个元素】

- **查找节点**
  - 以下方法可以找到若干个元素节点
    - document.getElementsByTagName()
    - document.querySelector('选择器'); // 根据指定选择器返回第一个元素对象
    - document.querySelectorAll('选择器'); // 根据指定选择器返回所有元素
  - TagName、All方法会得到一个伪数组，找到几个元素长度就为几
  - querySelector 和 querySelectorAll里面的选择器需要加符号,比如:document.querySelector('#nav');
  - 伪数组
    - 能有序存多个数据，且有下标、有长度，但没有真数组方法（例如push）的一种数据
  - 遍历节点
    - 因为伪数组也有下标、长度，所以也可以用原来的for循环进行遍历
    - 例：给每个li标签设置文字红色

- **更新元素样式**
  - 元素.style.样式名 = 样式值
    - 缺点：如果要同时设置多个样式会比较繁琐
  - cssText
    - 通过cssText属性，可以一次设置多个样式
    - 缺点：如果这类样式多个元素要用，改动麻烦
  - className
    - 本质是先写好类样式，再通过className修改元素拥有这个类
    - 缺点：需要预先写样式

- **获取元素样式**
  - 使用getComputedStyle、getPropertyValue两个方法
  - 参考：Window.getComputedStyle()

- **自定义属性**
  - 在 HTML 中除了标签的【标准属性】外，开发者还可以给标签【自定义属性】
    - 例：img的标准属性为src、alt等，但若写一个 data-info 则data-info叫自定义属性
  - 作用：一般用来存储额外数据辅助完成某个功能
  - 规范：自定义属性一律以 data- 开头，后面接名字，且名字中若多个单词用-隔开
    - 例：data-info 、data-label、data-login-name等
    - 理由：方便区分什么是【标准属性】，什么是【自定义属性】
  - JavaScript操作自定义属性：dataset
    - 元素.dataset.自定义属性名
    - 注意：不用加data-，后面遵守驼峰命名法
    - 例：某元素的自定义属性叫 data-label，则元素.dataset.label
    - 通过dataset可以获取自定义属性也可以重新赋值，若赋值的自定义属性不存在时则自动添加